C PROGRAMMING | PRACTICE QUESTION BANK

# Structures & Unions in C

Beginner to intermediate | with a hint for every question

| 22 | 5 | 22 | 14 |
|---|---|---|---|
| Practice questions | Sections | Hints included | Easy questions |

## Contents


| | | |
|---|---|---|
| A | **Structure Basics**<br>Declaring, reading, printing and copying structures. | A1 - A5 |
| B | **Arrays of Structures and Functions**<br>Handling many records and passing structures to functions. | B1 - B5 |
| C | **Nested Structures and Pointers**<br>A structure inside a structure, and the arrow operator. | C1 - C4 |
| D | **Unions**<br>Shared memory, sizeof, and when to use a union. | D1 - D5 |
| E | **Mixed Practice**<br>Spot the errors, then put it all together. | E1 - E3 |

## How to use this set

| | |
|---|---|
| 1 | **Attempt first.** Spend 10 to 15 minutes on a question before reading its hint box. |
| 2 | **Use any C compiler** you like (Code::Blocks, Dev-C++, VS Code or an online compiler). Type the program and run it. |
| 3 | **Predict, then run.** For output and size questions, write your answer on paper before running. |
| 4 | **Check yourself** with the answer key and revision notes on the last page. |

DIFFICULTY EASY MEDIUM CONCEPT Concept = explain in words, little coding.

Assume `char` = 1, `int` = 4, `float` = 4 bytes. Keep names to a single word (no spaces) so `scanf` stays simple.

# A Structure Basics

Declaring, reading, printing and copying structures.

### A1 Student Details — EASY

Define `struct Student { int roll; char name[30]; float marks; }`. Read the details of one student and display them.

- Print "Pass" if marks are 40 or more, otherwise "Fail".
- Display the details in a neat format, for example `Roll: 1 Name: Ravi Marks: 78.50`.

**HINT** Declare a variable like `struct Student s;` and reach its members with the dot operator: `s.roll`, `s.name`, `s.marks`. Use `%d` for roll, `%s` for name (no `&` for a character array) and `%f` for marks. Keep the name to a single word while practising.

### A2 Book with Discount — EASY

Create `struct Book` with title, author and price. Read one book and display it.

- Apply a 10% discount and print the new price.
- Read a second book and print the title of the costlier one.

**HINT** Discounted price = `b.price - b.price * 10 / 100` (or `b.price * 0.9`). For the second task you need two variables, `b1` and `b2`, and an `if` comparing `b1.price` with `b2.price`.

## A3 Initialising and Copying — EASY

Define `struct Point { int x, y; }` and initialise `struct Point p1 = {3, 4};`.

- Copy `p1` into a new variable `p2` using `p2 = p1;`.
- Change `p2.x` to 10 and print both points. Is `p1` affected?
- Print the sum of coordinates of each point.

> **HINT** Assigning one structure variable to another copies **all** its members at once, but the two variables are then independent. Changing `p2` will not change `p1`.

## A4 Predict the Output — EASY

Study the program below.

```
#include <stdio.h>
struct Item { int qty; float price; };

int main(void) {
    struct Item a = {5, 20.5};
    struct Item b = a;
    b.qty = 10;
    printf("%d %.1f\n", a.qty, a.price);
    printf("%d %.1f\n", b.qty, b.price);
    return 0;
}
```

- Write down the two output lines, then run the program and check.

> **HINT** `b = a` makes a copy. After that, changing `b.qty` only changes `b`. Trace the values of `a` and `b` line by line.

## A5 How Big is a Structure? — MEDIUM

Print `sizeof` of each structure below and compare it with the sum of the sizes of its members.

```
struct X { int a; float b; };
struct Y { char name[10]; int age; };
```

- Which one gives exactly the sum? Which one gives something bigger?
- Try changing `name[10]` to `name[12]` in `Y` and see what happens to the size.

> **HINT** Compilers sometimes add a few unused bytes (called **padding**) so that members sit at convenient memory addresses. Assume `int` = 4 bytes, `float` = 4 bytes, `char` = 1 byte. Add the member sizes first, then compare with what the program prints.

# B Arrays of Structures and Functions

Handling many records and passing structures to functions.

## B1 Highest Marks — EASY

Read the roll number, name and marks of `n` students (`n <= 10`) into an array of structures.

- Find the student with the highest marks and print their name and marks.
- Print the average marks of the class.

**HINT** Declare `struct Student s[10];` and use a `for` loop with `s[i].marks`. To find the highest, keep a variable `maxIndex = 0` and update it whenever `s[i].marks > s[maxIndex].marks`.

## B2 Employee Salary Report — EASY

Use `struct Employee { int id; char name[20]; float salary; }` and read 5 employees.

- Find the average salary.
- Print only those employees whose salary is **above** the average.

**HINT** You need two loops. The first loop adds up all salaries to get the average. The second loop goes through the array again and prints a record only when `e[i].salary > avg`.

## B3 Sort Students by Marks — MEDIUM

Read `n` students into an array of structures and arrange them in descending order of marks.

- Use bubble sort (or any sort you know).
- Print the sorted list as a rank list.

**HINT** Sorting works exactly like sorting numbers, except you compare `s[j].marks` and `s[j+1].marks`. To swap whole records use a temporary structure:
`struct Student temp = s[j]; s[j] = s[j+1]; s[j+1] = temp;`

## B4 Passing a Structure to a Function — EASY

Each student has three subject marks: `struct Student { char name[20]; int m1, m2, m3; }`.

- Write `float average(struct Student s)` that returns the average of the three marks.
- Write `void display(struct Student s)` that prints the name, total and average.
- Call both functions from `main` for every student in an array of 3 students.

**HINT** A structure is passed to a function just like an ordinary variable. The function receives a **copy**. Write the function above `main` (or put a prototype at the top). Remember to divide by `3.0` and not `3` so the average is not rounded down.

### B5 A Function that Changes the Structure

MEDIUM

Write `void giveBonus(struct Employee *e)` that increases the salary by 10%.

- Call it from `main` using `giveBonus(&emp);` and print the salary before and after.
- Now write a version that takes `struct Employee e` (without `*`). Does the salary change in `main`? Why not?

**HINT** When you pass a pointer to a structure, use the arrow operator: `e->salary`. It means exactly the same as `(*e).salary`. A function that receives a copy can only change the copy, not the original.

## C Nested Structures and Pointers

A structure inside a structure, and the arrow operator.

### C1 Person with Date of Birth

EASY

Define `struct Date { int d, m, y; }` and `struct Person { char name[20]; struct Date dob; }`.

- Read the details of one person and print them as `Name: Asha DOB: 12/5/2004`.
- Read 3 persons and print the name of the youngest.

**HINT** Reach the inner members with two dots: `p.dob.d`, `p.dob.m`, `p.dob.y`. For the youngest, compare the year first; if years are equal compare months, and so on.

### C2 Student with Marks Structure

EASY

Define `struct Marks { int m1, m2, m3; }` and `struct Student { int roll; char name[20]; struct Marks mk; }`.

- Read the details of 3 students.
- Print the total and percentage of each student.

**HINT** Marks are inside the student, so you write `s[i].mk.m1`, `s[i].mk.m2`, `s[i].mk.m3`. Total = sum of the three; percentage = `total / 3.0` if each subject is out of 100.

## C3 Predict the Output: Pointer to Structure

EASY

Study the program below.

```
#include <stdio.h>
struct Rect { int w, h; };

int main(void) {
    struct Rect r = {4, 5};
    struct Rect *p = &r;
    p->w = 10;
    printf("%d\n", r.w);
    printf("%d\n", (*p).h);
    printf("%d\n", p->w * p->h);
    return 0;
}
```

- Write the three output lines, then run to check.
- Why can't we write `*p.w` instead of `(*p).w`?

**HINT** `p` holds the address of `r`, so changing `p->w` changes `r.w` itself. For the last part, remember that the dot operator has higher priority than `*`, so `*p.w` is read as `*(p.w)`.

## C4 Walking Through an Array with a Pointer

MEDIUM

Create `struct Point { int x, y; }` and an array of 3 points with values of your choice.

- Declare `struct Point *p = pt;` and print all three points using `p->x` and `p->y` inside a loop.
- Do it twice: once using `p++`, and once using `(p + i)->x`.

**HINT** The name of an array is the address of its first element, so `p = pt;` is correct (no `&`). `p++` moves the pointer to the next structure in the array automatically. If you use `p++` inside the loop, reset `p = pt;` before the second version.

# D Unions

Shared memory, sizeof, and when to use a union.

## D1 Your First Union

EASY

Declare `union Data { int i; float f; char c; };` and a variable `union Data d;`.

- Store `d.i = 10;` and print `d.i`.
- Now store `d.f = 3.5;` and print `d.f`, then print `d.i` again. What do you notice?
- Explain in one or two lines why this happens.

**HINT** All members of a union share the **same memory location**. Storing a new value overwrites the old one, so only the member you stored most recently holds a meaningful value. Reading another member gives an unpredictable-looking number.

## D2 sizeof: Structure vs Union

EASY

Take the same three members in both a structure and a union.

```
struct S { int i; float f; char c; };
union  U { int i; float f; char c; };
```

- Print `sizeof(struct S)` and `sizeof(union U)`.
- Explain why the sizes are different.

**HINT** A structure stores every member separately, so its size is about the sum of all members. A union only needs space for its **largest** member, because all members share that space. Assume `int` = 4, `float` = 4, `char` = 1.

## D3 Predict the Output: Union

EASY

Study the program below.

```
#include <stdio.h>
union Num { int a; int b; };

int main(void) {
    union Num n;
    n.a = 25;
    printf("%d\n", n.b);
    n.b = 40;
    printf("%d\n", n.a);
    return 0;
}
```

- Write down both output lines, then run to check.

**HINT** `a` and `b` are two names for the same memory. Whatever you store through one name can be read through the other.

## D4 Product Code: Number or Text

MEDIUM

A shop's product code is either a number (like `1045`) or a short text (like `AB12`). Use `union Code { int num; char text[10]; }`.

- Keep a separate variable `type` (1 = number, 2 = text) that remembers what was stored.
- Read the type and the code from the user, then print the code correctly using `type`.
- Why do we need the extra `type` variable?

**HINT** A union cannot tell you which member was stored last, so you must remember it yourself. Use an `if (type == 1)` to print `c.num` with `%d`, and `else` to print `c.text` with `%s`. Later you can put both `type` and the union inside one structure.

## D5 Structure vs Union: Compare

CONCEPT

Make a table comparing a structure and a union on these points, and then give one example where each is useful.

- Keyword used, memory allocation, `sizeof`, how many members hold a valid value at the same time.
- One real example where a union saves memory.

**HINT** Think of a structure as a **cupboard with separate shelves** (everything stored at once) and a union as **one shelf** that holds only one item at a time. A union is useful when only one of several possible values is needed at a time, like the product code in D4.

# E Mixed Practice

Spot the errors, then put it all together.

## E1 Find the Error

EASY

Each snippet has a mistake. Find it and write the corrected line.

```
/* (a) */ struct S { int x; float y; }
           struct S s1;

/* (b) */ struct Student { char name[20]; } st;
           st.name = "Ravi";

/* (c) */ struct S a = {1, 2.5}, b = {1, 2.5};
           if (a == b) printf("Same");

/* (d) */ struct S { int x; };
           S s1;
```

- Explain each mistake in one line and give the correct code.

**HINT** (a) Look at the end of the structure definition. (b) You cannot assign to a character array with `=`; think of `strcpy` from `string.h`. (c) Structures cannot be compared with `==`; compare member by member. (d) In C, the word `struct` is needed when declaring the variable (unless you use `typedef`).

## E2 Mini Library Menu

MEDIUM

Use `struct Book { int id; char title[30]; float price; }` and an array that can hold 5 books.

- Show a menu: 1. Add book, 2. Display all, 3. Search by id, 4. Exit.
- Keep a counter `count` of how many books are stored and stop adding when the array is full.
- In search, print "Not found" if the id does not exist.

**HINT** Use a `do ... while` loop around a `switch`. Adding a book means reading into `b[count]` and then doing `count++`. For search, loop from `0` to `count - 1` and compare `b[i].id` with the id entered. Use a flag variable (`found = 0`) to decide whether to print "Not found".

## E3 Result Card

MEDIUM

Use `struct Marks { int m1, m2, m3; }` and `struct Student { int roll; char name[20]; struct Marks mk; float percent; char grade; }`. Read `n` students (`n <= 5`).

- Calculate the percentage of each student and store it in the structure.
- Assign a grade: A (75 or more), B (60 to 74), C (40 to 59), otherwise F.
- Print a table of roll, name, percentage and grade, then print the topper.

**HINT** Do it in steps: first read, then calculate, then print. Use `if ... else if` for grades. For the topper, keep the index of the student with the highest percentage while you loop, exactly as in B1.

# Answer Key and Revision Notes

Use this only after you have attempted the questions.

## Output and size questions

| | |
|---|---|
| **A4** | `5 20.5` and `10 20.5` |
| **A5** | `X` = 8 bytes (4 + 4). `Y` = 16 bytes on most machines, not 14, because of 2 padding bytes. |
| **C3** | `10`, `5`, `50` |
| **D1** | The first two prints are correct. The last one prints a strange number (for 3.5 it is usually 1080033280) because the float overwrote the int. |
| **D2** | `sizeof(struct S)` = 12 on most machines, `sizeof(union U)` = 4. |
| **D3** | `25` and `40` |

## Quick revision notes

- A **structure** gives every member its own memory; a **union** shares one block between all members.
- Use `.` with a structure variable and `->` with a pointer to a structure: `p->x` means `(*p).x`.
- One structure variable can be assigned to another with `=`, but two structures cannot be compared with `==`.
- To change a structure inside a function, pass its address (a pointer). Passing by value only changes a copy.
- For nested structures, chain the dots: `s.dob.year`.
- A union's size equals its largest member. Only the last stored member holds a valid value.
- Use a separate variable (a "tag") to remember which union member is currently in use.
- A character array cannot be assigned with `=`; use `strcpy`.
- Optional: `typedef struct { ... } Name;` lets you write `Name x;` without the word `struct`.